Java Silver（SE 17・1Z0-825-JPN）　3 演算子と制御構造　#06

分岐と switch

if の範囲はかっこで決まり、switch はコロンなら落ちていく ― if・else、switch 文、アロー形式、switch 式と yield

各ページの下に、根拠にした Oracle の資料（JLS 14 章・15.28）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 3（if/else、switch 文、switch 式）

1 if 文
対象は直後の 1 文。else はいちばん近い if に付く。条件は boolean

2 switch 文
セレクタの型と case 定数の決まり。break がないと下へ落ちる

3 アロー形式と switch 式
-> は落ちない。switch 式は値を返し、すべての値を処理する

	コロン（:）	アロー（->）
フォールスルー	する（break で止める）	しない
switch 文の右側	文をいくつでも	式文・ブロック・throw
switch 式の値	`yield 値;`	`-> 値;` か `yield`
変数のスコープ	switch の { } 全体	各ブロックの中
まぜて書く	different case kinds used in the switch	

	switch 文	switch 式
値	返さない	返す（最後に ;）
すべての値	処理しなくてよい	**必ず処理する**（網羅性）

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 14.11 The switch Statement／JLS 15.28 switch Expressions（all possible values of that expression must be handled）

第1章

if 文

字下げは飾り。範囲を決めるのは { } と文の区切り。

if の範囲とぶら下がり else

else はいちばん近い if に付く（Claude が作成）

```
int x = 5;
if (x > 3)
  if (x > 10) System.out.println("A");
else System.out.println("B");   // 内側の if のもの
System.out.println("C");
```

```
B
C
```

コンパイラから見た形（かっこを補った）

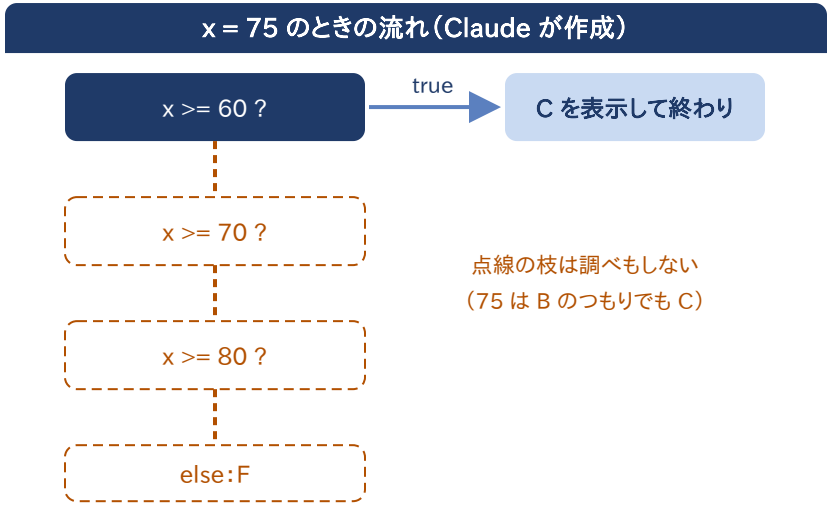
```
if (x > 3) {
  if (x > 10) { …"A" } else { …"B" }
}
System.out.println("C");
```

書き方	結果（JDK 17）
{ } なしで 2 行を字下げ	2 行目は if の外（いつも実行）
`if (…) A; B; else C;`	'else' without 'if'
`if (x > 0) ; { … }`	{ } はいつも実行（; が空文）
`if (x = 10)`（x は int）	int cannot be converted to boolean
`if (b = x > 0)`（b は boolean）	OK（代入式の値が条件）
`Boolean f …; if (f)`	OK（アンボクシング）
`else (x < 0) { }`	not a statement（else if にする）

読み方：if のあとの**文 1 つ**（または { } 1 つ）だけが対象。外側の if に else を付けたいときは、内側の if を { } で囲む。

出典：JLS 14.5 Statements（an else clause belongs to the innermost if to which it might possibly belong）・14.6 The Empty Statement（An empty statement does nothing）・14.9 The if Statement（The Expression must have type boolean or Boolean）／結果は JDK 17 で確かめたもの

else if は上から 1 つだけ




```
// 正しい順：範囲のせまい条件から
if (x >= 80)      System.out.println("A");
else if (x >= 70) System.out.println("B");
else if (x >= 60) System.out.println("C");
else               System.out.println("F");
```

else if は特別な文法ではない。else のあとの文が、たまたま次の if 文になっているだけ。だから上から順に、最初に true になった枝だけを実行する。

else だけの枝：上のどれにも当てはまらないとき。else のあとに条件は書けない。

出典：JLS 14.9.2 The if-then-else Statement（If the value is true, then the first contained Statement … is executed／If the value is false, then the second contained Statement … is executed）／結果は JDK 17 で確かめたもの

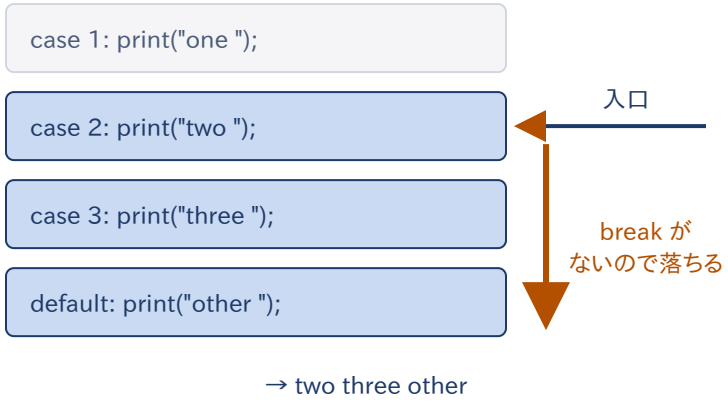
第2章

switch 文

合った case から入り、break まで下へ落ちていく。

入口は 1 つ、出口は break

k = 2 のとき（Claude が作成）



ケース	結果（JDK 17）
上の switch で k = 2	**two three other**
k = 9（合う case なし）	**other**
default を先頭に置き、k = 9、その次の case 1 に break	**d 1（default から break まで）**
合う case も default もない	**何もしない**

> **default は「合う case がないときの入口」。**どこに書いても、合う case があれば使われない。入ったあとは break まで落ちる。1 つの switch に 1 つまで。

出典：JLS 14.11.3 Execution of a switch Statement（all the statements in the switch block that follow the switch label are executed in order／If no switch label matches, the entire switch statement completes normally／例 14.11.3-1 Fall-Through in the switch Statement）・14.11.1（If no case label matches but there is a default label …）／結果は JDK 17 で確かめたもの

セレクタの型と case 定数の決まり

セレクタの型	使えるか
char・byte・short・int	使える
Character・Byte・Short・Integer	使える（アンボクシング）
String・enum	使える
long・float・double・boolean	コンパイルエラー

long や boolean を使ったときの JDK 17 の表示は「patterns in switch statements are a preview feature …」と、もう一つの型のエラー。

case の書き方	結果（JDK 17）
`case y:`（y はふつうの変数）	constant expression required
`final int y = 2; … case y:`	OK（定数変数）
`case 1: … case 1:`	duplicate case label
byte のセレクタに `case 200:`	possible lossy conversion
char のセレクタに `case 98:`	OK（'b' のとき合う）
`case 1, 7:`	OK（どちらかに合えば）

case 定数は「コンパイル時に値が決まる」もの：リテラル、定数式、final で定数に初期化した変数、enum の定数名。セレクタの型に代入できる値であること。

出典：JLS 14.11（The type of the selector expression must be char, byte, short, int, Character, Byte, Short, Integer, String, or an enum type）・14.11.1（Every case constant must be either a constant expression or the name of an enum constant／No two of the case constants … may have the same value／every case constant … is assignment compatible with T）／JLS 15.29／結果は JDK 17 で確かめたもの

String・enum・null とスコープ

コード	結果（JDK 17）
s が "Java" で `case "java":`	合わない（equals で比べる）
s が null の switch	NullPointerException（default にも入らない）
`case Color.RED:`	must be the unqualified name of an enumeration constant
`case RED:`	OK
switch 文で enum の定数が足りない	OK（合わなければ何もしない）

case null は書けない（Java 17）。セレクタが null になりうるときは、switch の前に null を調べる。

コロン形式の { } は 1 つのブロック（Claude が作成）

```
int k = 3;
switch (k) {
  case 1:
    int a = 10;      // ここで宣言
    break;
  case 3:
    a = 30;          // 見える。代入してから使う
    System.out.println(a);   // 30
    break;
}
```

アロー形式で `case 1 -> { int a = 10; }` と書けば、a はそのブロックの中だけ。各ブロックで同じ名前を宣言できる。

出典：JLS 14.11.1（unless the value is a String, in which case equality is defined in terms of the equals method of class String／If T is an enum type, then every case constant … is an enum constant of type T）・14.11.3（if the result of evaluating the selector expression is null, then a NullPointerException is thrown）／JLS 6.3（the rest of the block）／結果は JDK 17 で確かめたもの

第3章

アロー形式と switch 式

-> は落ちない。switch 式は必ず値を返す。

アロー形式（switch ルール）

```
int day = 6;
switch (day) {
  case 1, 7 -> System.out.println("holiday");
  case 2, 3, 4, 5, 6 -> System.out.println("weekday");
  default -> System.out.println("?");
}
// weekday だけ（下へ落ちない）
```

右側の形	例
式	`case 1 -> count++;`
ブロック	`case 1 -> { … }`
throw 文	`case 1 -> throw new …;`

書き方	結果（JDK 17）
`case 1 -> …; case 2: …`	different case kinds used in the switch
switch 文で `case 2 -> "two";`	not a statement
`default ->` を先頭、k は 2	case 2 だけ実行
ブロックごとに `int a` を宣言	OK（スコープが別）

> **switch 文の -> の右は「式文」だけ：**メソッド呼び出し・代入・++/-- ・new。値だけの式（"two" や 1 + 2）は文にならない。

出典：JLS 14.11.1（SwitchRule: SwitchLabel -> Expression ; | SwitchLabel -> Block | SwitchLabel -> ThrowStatement）・14.11.2（Every switch rule expression in the switch block is a statement expression）・14.11.3（the switch rule expression or statement introduced by the switch label is executed, and nothing else）・14.8 Expression Statements／結果は JDK 17 で確かめたもの

switch 式 ― 値を返し、すべての値を処理する

```
String s = switch (day) {
  case 1, 7 -> "holiday";
  default -> {
    System.out.print("calc ");
    yield "weekday";     // ブロックから値を返す
  }
};                              // 代入文の終わりの ;
```

ブロックの最後に書けないもの（JDK 17 の表示）

`"weekday";`	not a statement
`return "weekday";`	attempt to return out of a switch expression
`break "weekday";`	';' expected

セレクタ	網羅性の満たし方
int・String など	**default が必須**（ちょうど 1 つ）
enum	全定数を case に書けば default 不要

足りないと：the switch expression does not cover all possible input values。実際に来る値が書いてあっても、型で判断される。

switch 文との差：switch 文は、合うものがなければ何もせず終わる。switch 式は必ず値が必要なので、コンパイル時に全部を処理しているか調べる。

出典：JLS 15.28.1 The Switch Block of a switch Expression（If the type of the selector expression is not an enum type, then there is exactly one default label／If the type … is an enum type, then (i) the set of the case constants … includes every enum constant … A default label is permitted, but not required）／JLS 14.21 The yield Statement／結果は JDK 17 で確かめたもの

コロン形式の switch 式・式の型・throw

コロン形式でも書ける（yield まで落ちる）

```
int k = 2;
int r = switch (k) {
  case 1: yield 10;
  case 2: System.out.print("x ");
  case 3: yield 30;
  default: yield 0;
};
System.out.println(r);          // x 30
```

throw の枝

```
int r = switch (k) {
  case 2 -> throw new IllegalStateException("no");
  default -> 1;
};                              // k が 2 なら例外
```

結果の式	switch 式の型（var で受けたとき）
全部 String	String
int と double	double（1 は 1.0）
boolean と Boolean	boolean
String と int	共通の参照型（Object など）

```
var r = switch (1) {
  case 1 -> 1;  case 2 -> 2.5;  default -> 0;
};   // 1.0
```

代入先がある場合（`int r = switch…`）は、各結果の式が int に代入できればよい。

出典：JLS 15.28.1（If the switch block consists of switch labeled statement groups, then every expression immediately contained in a yield statement … is a result expression／The type of a standalone switch expression … general numeric promotion／The type of a poly switch expression is the same as its target type）／JLS 14.11.1（SwitchLabel -> ThrowStatement）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
if の表示結果	字下げを無視し、{ } と ; で範囲を読む。else はいちばん近い if
else if の結果	上から順に、最初に true の枝だけ
コロン形式の switch の表示	合った case から break まで。default は合う case がないときの入口
switch がコンパイルできるか	セレクタの型、case は定数・重複なし・型に入る値、enum は定数名だけ、: と -> をまぜない
switch 式がコンパイルできるか	網羅性（default か enum 全定数）、ブロックは yield、最後の ;
例外になるか	セレクタが null なら NullPointerException。throw の枝
switch 式の値	アローは落ちない。コロン形式は yield まで落ちる。int と double がまざれば double

出典：このPDFの各ページの資料

まとめ

if 文
対象は 1 文。else は近い if。条件は boolean か Boolean

switch 文
セレクタは int 以下・ラッパー・String・enum。break まで落ちる

アロー形式と switch 式
-> は落ちない。switch 式は網羅して値を返す。yield

こんなとき	結果
break のない case 2（k は 2）	**下の case も実行**
`switch (1L)`	**コンパイルエラー**
String のセレクタが null	**NullPointerException**
int の switch 式に default なし	**コンパイルエラー**
ブロックから値を返す	**yield 値;**

次は #07 繰り返し。while・do-while・for・拡張 for と、break・continue・ラベルへ進む。

出典：このPDFの各ページの資料